

Health Wearable Device

Firmware Architecture & Code Analysis Report



Band01_Rev1 · Project Work — Task 2

Prepared by: Masrur Jamil Prochchhod
Institution: Hochschule Hamm-Lippstadt (HSHL)
Matriculation: 2210028
Date: 27 May 2026

70

Source Files

8

Directories

1,647

Lines (main)

13

Report Sections

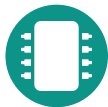
4

RTOS Tasks



What Is This Firmware?

The firmware is the software that runs directly on the CC2640R2F microcontroller — it is the "brain" of the smartband.



It is NOT a phone app

It runs on bare metal inside the wristband chip itself, not on Android/iOS. There is no operating system like Windows — only a lightweight RTOS (SYS/BIOS).



It talks to hardware directly

Via I²C, it reads MAX30102 (heart rate), MAX30205 (temperature) and LIS331 (motion). It controls every GPIO pin on the PCB.



It transmits data wirelessly

Every 350 ms, sensor data is packaged into a 6-byte BLE GATT notification and sent to a smartphone or hospital gateway via BLE 5.1.



It manages power

It controls the LP5907 regulator EN pins to power down sensors, and uses deep sleep between BLE events to achieve 73-hour battery life.

Built on: TI SimpleLink CC2640R2 SDK · SYS/BIOS RTOS · TI BLE Stack · Code Composer Studio v10+

Teaching note — Unlike desktop software, embedded firmware has no loader — it boots directly from flash memory. Every line of code must fit within 128 KB total flash on this chip.



Development Environment & Repository Structure

Required Tools & SDKs

CCS v10+	Eclipse-based TI IDE — compile, link, flash, debug
SimpleLink SDK	BLE stack + hardware drivers for CC2640R2F
TI ARM Compiler	Included with CCS — targets ARM Cortex-M3
BLE Stack (ROM)	Pre-compiled — stored in FlashROM_StackLibrary/
SYS/BIOS RTOS	Real-time OS — part of TI SDK
XDS110 Probe	USB JTAG debugger → connects to P1 header on PCB

Repository Folder Map

Startup/	Boot, CCFG, RTOS entry point
Application/	Sensors, OLED, BLE app logic
PROFILES/	BLE GATT service definitions
Include/	Shared header files
FlashROM_Stack/	Pre-compiled BLE stack (do not edit)
TOOLS/	SYS/BIOS config (app_ble.cfg)
targetConfigs/	CCS debug target for CC2640R2F
BOM/	Aisler Bill of Materials PDF

Teaching note — CCS is mandatory for compilation. Source files can be read in VS Code, but building, flashing and SDK path configuration must be done inside CCS.



Boot Sequence — Startup/main.c

8 Steps before BIOS_start()

- 1 RegisterAssertCbak()**
Register BLE error handler
- 2 PIN_init()**
Initialise all GPIO pins
- 3 VIMSMODESet()**
Enable instruction cache
- 4 ICall_init()**
Set up inter-task messaging
- 5 ICall_createRemoteTasks()**
Start BLE stack (Priority 5)
- 6 GAPRole_createTask()**
Create GAP role (Priority 5)
- 7 SimplePeripheral_createTask()**
Create app task (Priority 2)
- 8 BIOS_start()**
Hand control to RTOS — never returns

```
int main() {  
    RegisterAssertCbak(AssertHandler);  
    PIN_init(BoardGpioInitTable);  
    ICall_init();  
    ICall_createRemoteTasks(); // BLE Pri 5  
    GAPRole_createTask();    // GAP Pri 5  
    SimplePeripheral_createTask(); // App Pri 2  
    //App_createTask();      // DISABLED  
    //OledThread_createTask(); // DISABLED  
    BIOS_start(); // never returns  
}
```



AppThread & OledThread exist in the repo but are commented out in main.c — the code is ready, just not yet activated.

ccfg_app_ble.c — chip-level flash config (bootloader, power, radio trim).
Compiled separately, never modified by user.

Teaching note — BIOS_start() is the point of no return — the RTOS scheduler takes over. Priority 5 BLE tasks will always pre-empt the Priority 2 app task when radio events occur.



How the Main App Task Works — SimplePeripheral



Teaching note — The 350 ms period was chosen to match the research paper's power budget of 2.032 mA average. Shorter interval = more radio activity = shorter battery life.



simple_peripheral.c — Core Sensor & BLE Loop

SmartBand_periodicFxn() — runs every 350 ms

```
// Fires every 350 ms via SYS/BIOS Clock
void SmartBand_periodicFxn(void) {
    GAPRole_GetParameter(GAPROLE_STATE, &state);
    if (state == GAPROLE_CONNECTED) {
        i2c_signal_data = 0x7788; // ← placeholder
        // ACTIVATE: get_i2c_value_max30205_rawtemp()
        uint8_t arr[] = {0,1,2,0,upper,lower};
        SimpleProfile_SetParameter(
            SIMPLEPROFILE_CHAR1, 6, arr);
    }
}
```

Button handling — buttonCallbackFxn()

```
// GPIO interrupt — fires on button press
void buttonCallbackFxn(PIN_Handle h, PIN_Id id) {
    PIN_setConfig(h, PIN_BM_IRQ, id | PIN_IRQ_DIS);
    if (id == Board_BUTTON0) button0State ^= 1;
    if (id == Board_BUTTON1) button1State ^= 1;
    // Re-enabled at end of periodicFxn()
}
```

What each part does

GAPRole_GetParameter()

Checks if a phone is currently connected. Sensor data is only sent when BLE is connected — saving power when no phone is nearby.

0x7788 placeholder

The current firmware transmits this hardcoded value instead of real sensor data. It proves BLE works before sensors are enabled.

SimpleProfile_SetParameter()

Writes the 6-byte array into GATT CHAR1 (UUID 0xFFFF1). The TI BLE stack then automatically sends a GATT notification to the phone.

buttonCallbackFxn()

Runs in interrupt context (not RTOS task context). Interrupt is disabled on entry to debounce — re-enabled 350 ms later by the clock.

Teaching note — SimplePeripheral_init() is called once at boot to register GATT, configure BLE params, set up button interrupts, and draw the OLED splash screen.



I²C Sensor Driver — i2c_sensors.c (437 lines)

Standard I²C Read Pattern

```
I2C_Params params;
I2C_Transaction txn;
uint8_t txBuf[1] = {REG_ADDR};
uint8_t rxBuf[1];
params.bitRate = I2C_400kHz;
txn.slaveAddress = 0x57; // MAX30102
txn.writeBuf=txBuf; txn.writeCount=1;
txn.readBuf=rxBuf; txn.readCount=1;
handle = I2C_open(Board_I2C0, &params);
if (I2C_transfer(handle, &txn))
    return rxBuf[0];
I2C_close(handle);
```

Temperature conversion (MAX30205):

```
uint16_t raw = (rxBuf[0]<<8 | rxBuf[1]);
float degC = raw * 0.00390625f; // 1/256°C
```

Sensor Functions Available

MAX30102	0x57	get_i2c_value_max3010x_id() Read PART_ID — chip health check
MAX30102	0x57	get_i2c_value_max3010x_basicinit() Set LED current to 6.2 mA
LIS331	0x18	get_i2c_value_lis331_powerup() Enable all axes, normal power
LIS331	0x18	get_i2c_value_lis331_whoami() Returns 0x33 (verify vs datasheet!)
LIS331	0x18	get_i2c_value_lis331_xh() / xl() X-axis acceleration high/low byte
MAX30205	0x48	get_i2c_value_max30205_rawtemp() Raw 16-bit temperature register



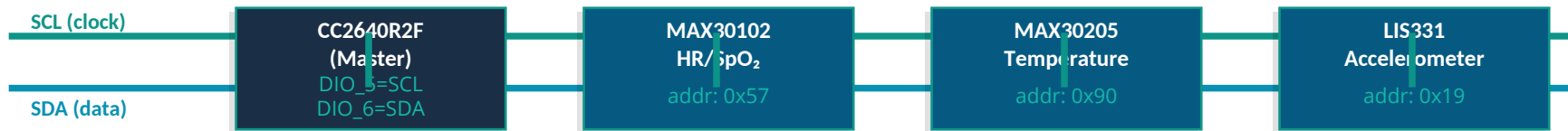
⚠ WHO_AM_I discrepancy: firmware uses 0x33 but LIS331DL datasheet specifies 0x3B. Verify the exact LIS331 variant on PCB before flashing — a mismatch causes silent sensor init failure.

Teaching note — All I²C functions open and close the bus per-transaction. In production, keep the handle open during sensor polling to reduce latency and overhead.



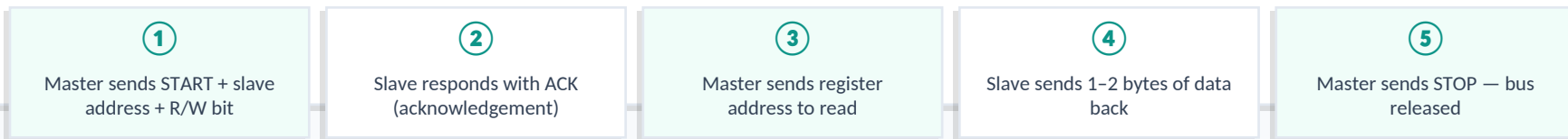
How I²C Communication Works — Explained

I²C (Inter-Integrated Circuit) — 2 wires, many devices, one master



R103/R104 — 10 k Ω pull-up resistors keep SCL and SDA HIGH when idle (I²C is open-drain — devices can only pull LOW)

A typical I²C read transaction — step by step:



Why I²C for this wearable?

Only 2 wires (SCL + SDA) connect all 3 sensors + OLED display — critical on a 34×40 mm PCB where every trace matters. The CC2640R2F is bus master, polling each sensor in turn using unique 7-bit addresses.

Teaching note — The 400 kHz fast-mode I²C used here can read 2 bytes from MAX30205 in ~40 μ s — fast enough for the 350 ms polling period while leaving the CPU free for BLE tasks.



OLED Display — oled.c + oled_thread.c

SSD1306 Driver — Public API (oled.c, 334 lines)

oledInit()

22-byte SSD1306 init sequence

oledFill(data)

Fill all 8 pages × 128 columns

oledWriteString(x,y,str)

Render ASCII text via ucFont[]

oledDrawPic(pic,w,h,x,y)

Binary bitmap from flash memory

oledSetPosition(x,y)

Move write cursor (page + column)

oledShutdown()

Send 0xAE — panel off (saves power)

oled_thread.c — Dedicated Task (183 lines)

Status: DISABLED in main.c — code complete, not activated

Priority: 1 — display is non-critical, always yields to BLE

Stack: 800 bytes

Pattern: Event-driven queue — waits on OLED_EVENT_SCREEN_UPDATE

Messages: MEDIC_PIC, HEART_PIC, BUTTON_PRESS, DATA_HEX

```
// Boot splash screen (currently active)
oledInit();
oledFill(0x00); // clear screen
oledWriteString(0,0,"Smart Band V001");
```

Once enabled: oled_thread displays live HR, SpO₂, temp and motion data in real time

Teaching note — Separating display into its own task (oled_thread) prevents OLED I²C transactions from blocking the BLE task — a core embedded multitasking principle.



BLE GATT Profile — Service 0xFFFF0

Five Characteristics — What the Smartphone Sees

Char	UUID	Length	Access	Current Use
CHAR1	0xFFFF1	6 bytes	R/W	Sensor array: [MAX3010x_id, LIS_whoami, LIS_xh, LIS_xl, temp_upper, temp_lower]
CHAR2	0xFFFF2	1 byte	Read	Status byte: bit[0]=button0, bit[4]=button1, BLE state
CHAR3	0xFFFF3	1 byte	R/W	General purpose — reserved for future sensor data
CHAR4	0xFFFF4	1 byte	Notify	Push-notification channel — phone receives alerts without polling
CHAR5	0xFFFF5	5 bytes	R/W	Extended buffer — reserved for future multi-byte values

End-to-End Data Flow — Sensor to Phone



Teaching note — GATT = Generic Attribute Profile. Any BLE scanner app (nRF Connect on Android/iOS) can connect and read CHAR1 live — no custom app needed during development.



Understanding BLE & GATT — How Wireless Data Works

BLE = Bluetooth Low Energy — designed for small data bursts from battery-powered devices, not continuous audio streaming like classic Bluetooth.

The CC2640R2F advertises every 100 ms ("I'm here!"). A phone connects, then receives GATT notifications every 350 ms containing the sensor data.

GATT Data Model — how data is organised:

Device (Band01_Rev1)

Service 0xFFFD (like a folder)

CHAR1 0xFFD1 — 6 bytes — sensor data array

CHAR2 0xFFD2 — 1 byte — button state

CHAR4 0xFFD4 — 1 byte — Notify (phone gets alerts without polling)

Teaching note — BLE GATT is like a USB stick with folders (Services) and files (Characteristics). The phone reads the 'file' (CHAR1) every 350 ms — no custom protocol or app code needed.



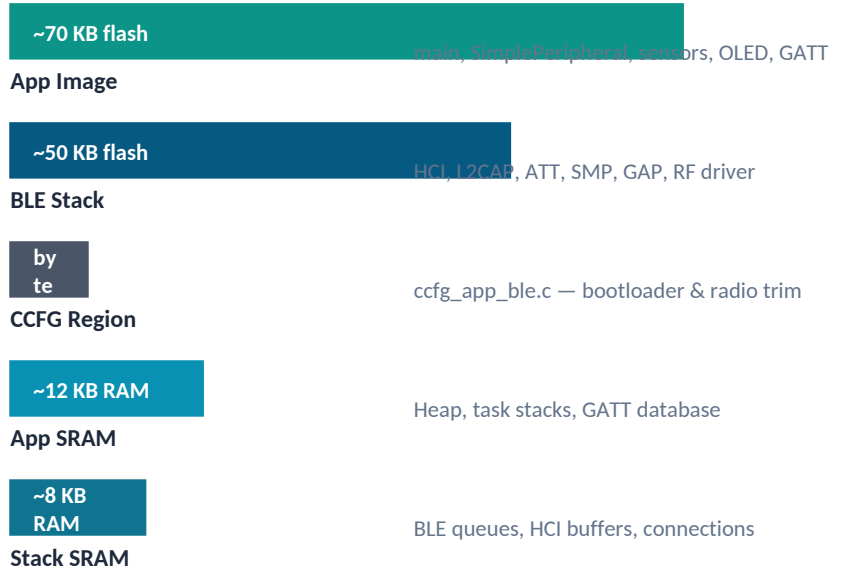
BLE Stack Layers & Memory Map

BLE Protocol Stack — FlashROM_StackLibrary/

Pre-compiled by TI • Do NOT modify

GAP	Generic Access Profile Advertising, scanning, connections
GATT	Generic Attribute Profile Service & characteristic layer
ATT	Attribute Protocol GATT data R/W/Notify at byte level
SMP	Security Manager Pairing, bonding, key exchange
L2CAP	Logical Link Control Channel mux, segmentation
HCI	Host Controller Interface App ↔ BLE radio bridge
RF	2.4 GHz Radio Driver CC2640R2F RF core control

CC2640R2F Memory Map (128 KB Flash / 20 KB SRAM)



Teaching note — App and BLE stack are two separately linked binaries sharing one 128 KB flash. Exact sizes are in Debug/smart_band_v001_app.map after a CCS build.



RTOS Architecture — SYS/BIOS Task Model

Task Table — Pre-emptive Fixed-Priority Scheduling

Task	Priority	Stack	Status	File	Responsibility	Notes
BLE Stack	5 ★	Internal	Active	FlashROM/	Radio, advertising, connections	Highest priority — always pre-empts app
GAPRole	5	Internal	Active	peripheral.c	GAP connection events	Handles BLE peripheral role
SimplePeripheral	2	800 B	Active	simple_perip...	Sensor reads + BLE updates	Core loop — every 350 ms
OledThread	1	800 B	Disabled	oled_thread.c	Event-driven display	Commented out — ready to enable
AppThread	1	700 B	Disabled	app_thread.c	Secondary sensor/signal	Scaffold — awaiting sensor integration

SYS/BIOS Primitives Used in This Firmware:

Clock

Periodic timers
(350 ms poll)

Event

Task sync
(OLED events)

Queue

Inter-task messages
(OLED queue)

Semaphore

Mutual exclusion
(resource locks)

Task

Thread definition
(each RTOS task)

Teaching note — SYS/BIOS uses fixed-priority pre-emptive scheduling — NOT POSIX. Priority 5 BLE tasks will always pre-empt the Priority 2 app task mid-execution when radio events occur.



Power Architecture in Firmware

Active / Connected

BLE: BLE connected + 350 ms notifications

Sensors: All sensors powered

~2.0 mA

Advertising

BLE: Periodic BLE adv (100 ms interval)

Sensors: Can be gated off

< 1 mA

Deep Sleep

BLE: 32 kHz Y2 crystal maintains schedule

Sensors: All powered down

< 1 μ A

Battery Life Calculation

150 mAh $\div$ 2.032 mA = 73.8 hours $\approx$ 73+ hours

Source: Pinheiro et al. (2021) — measured 2.032 mA avg at 350 ms BLE interval

POWER_SAVING compile flag:

Controls deep sleep permission. When enabled, MCU enters standby between BLE events. The 32 kHz crystal Y2 keeps waking it at precise intervals.

Teaching note — LP5907 EN pins are wired to CC2640R2F GPIOs — firmware can cut sensor power entirely during BLE advertising, achieving the <1 μ A standby current target.



Debugging — JTAG, Assertions & Recommended Improvements

Two Debug Paths

① JTAG via XDS110 probe + P1 header

Connect XDS110 USB probe to P1 header (18-pin)

· TCK → DIO_3 · TMS → DIO_4 · TDI + TDO

In CCS: set breakpoints, step through code line-by-line, inspect registers, watch variables live, measure current

② BLE Stack Assertion Handler

RegisterAssertCbback(AssertHandler) — registered in main()

Fires when BLE stack detects internal violations (wrong HCI state, message buffer exhaustion)

Current behaviour: loops forever — visible to debugger but silent to user without JTAG

Recommended Improvement — Visible Assert

Current AssertHandler loops silently. Improved version displays the error code on the OLED so faults are visible without a JTAG probe attached:

```
void AssertHandler(uint8 cause,
                  uint8 subcause) {
    char buf[24];
    oledFill(0x00);
    oledWriteString(0,0,"ASSERT ERROR");
    snprintf(buf,sizeof(buf),
             "cause=%02x sub=%02x",
             cause, subcause);
    oledWriteString(0,2,buf);
    while(1); // attach JTAG here
}
```

This is a recommended Task 3 improvement — diagnose faults in the field without a laptop.

Teaching note — JTAG debug is the primary tool during development. The assertion handler becomes essential once the device is deployed away from a desktop — it's your last line of defence.



Firmware–Hardware Pin Correspondence

Every DIO pin in firmware maps to a physical component on Band01_Rev1 (Task 1 reference)

HW Component	DIO	Code Ref	Code File	Purpose	Status
I ² C SCL (all sensors+OLED)	DIO_5	Board_I2C0 SCL	i2c_sensors.c, oled.c	I ² C clock 400kHz — shared by all 4 devices	Active
I ² C SDA (all sensors+OLED)	DIO_6	Board_I2C0 SDA	i2c_sensors.c, oled.c	I ² C data bidirectional	Active
MAX30102 interrupt	DIO_8	(not handled yet)	simple_peripheral.c	HR data-ready — not wired in firmware	Pending
LIS331 interrupt 1	DIO_9	(not handled yet)	simple_peripheral.c	Motion event interrupt 1	Pending
LIS331 interrupt 2	DIO_2	(not handled yet)	simple_peripheral.c	Motion event interrupt 2	Pending
SW101 Button	DIO_0	Board_BUTTON0	simple_peripheral.c	Falling-edge ISR, toggles state → CHAR2	Active
SW102 Button	DIO_1	Board_BUTTON1	simple_peripheral.c	Falling-edge ISR, toggles state → CHAR2	Active
JTAG TCK (P1 header)	DIO_3	JTAG TCK	targetConfigs/*.ccxml	CCS debug probe clock	Active
JTAG TMS (P1 header)	DIO_4	JTAG TMS	targetConfigs/*.ccxml	CCS debug mode select	Active
24 MHz Crystal Y1	X24M_P/N	BLE radio clock	FlashROM_Stack/	Required for BLE RF synthesis	Active

Teaching note — Interrupt pins DIO_8/9/2 are physically wired on the PCB but not yet handled in firmware. Activating them replaces polling with interrupt-driven reads — critical for power efficiency.

Current Status & Key Takeaways

Band01_Rev1 Firmware — Task 2 | Masrur Jamil Prochchhod | HSHL 2026

Implementation Status

✓ BLE advertising + connection	Complete
✓ GATT Service 0xFFFD (5 chars)	Complete
✓ OLED hardware driver	Complete
✓ Splash screen at boot	Complete
✓ Button GPIO interrupts	Complete
✓ I ² C driver for all sensors	Complete
✓ 350 ms periodic BLE update	Complete
⚠ Live MAX30205 temperature	Pending
⚠ Live MAX30102 HR / SpO ₂	Pending
⚠ Live LIS331 accelerometer	Pending
⚠ OLED live display	Pending
⚠ ESPRIT HR algorithm	Not started
⚠ BLE security hardening	Not started



CC2640R2F + SYS/BIOS

ARM Cortex-M3 + BLE 5.1 SoC running TI SYS/BIOS RTOS — 5 tasks, fixed-priority pre-emptive scheduling



Infrastructure Complete

BLE GATT, I²C drivers, OLED, button handling all done — 3 lines of code activate live sensors



Placeholder Active

Device currently transmits {0,1,2,0,0x77,0x88} — real sensor calls are commented out and ready



Dual-Binary Flash

App (~70 KB) + BLE stack (~50 KB) share 128 KB flash — memory map in .map file after CCS build



73-hour Battery Life

$2.032 \text{ mA avg} \times 150 \text{ mAh} = 73.8 \text{ h}$
POWER_SAVING flag + LP5907 EN pins enable <1 μA deep sleep



ESPRIT Algorithm Next

Research paper targets ESPRIT via Hilbert Transform for HR estimation at 100 Hz — not yet in firmware